SUPPORT ÉTUDIANT

DSP2 et Open Banking

Jour 2 : API, authentification et sécurité

Chrisvene MPANDZOU

chrisvene.mpandzou@a2mtech.com

M2 SIB · 2026 – 2027

Sommaire

Repères de lecture du polycopié

01 Introduction & objectifs 3

02 S1 · Architecture API 5

03 S2 · Workflow AIS 8

04 S3 · Workflow CBPII 12

05 S5 · Authentification & autorisation 16

06 S6 · Sécurité & conformité 23

Introduction & objectifs

ACCUEIL

Intro & objectifs

Le Jour 1 a posé le pourquoi et le qui. Le Jour 2 passe au comment technique : de quoi est faite une API bancaire, par quels mécanismes un tiers est autorisé, et comment l'ensemble reste sûr et conforme.

FIL ROUGE

Samedi, 9 h 12. L'application de Lyam prépare sa requête. Elle doit **prouver qui elle est** (le TPP), obtenir une **autorisation** (le consentement de Lyam), puis présenter un **jeton** à chaque appel. Le Jour 2 ouvre ce capot.

RAPPEL

Ce que l'on a fixé au Jour 1

> **DSP2** — **Deuxième Directive sur les Services de Paiement**, directive (UE) 2015/2366. Elle oblige les banques à ouvrir l'accès aux comptes et aux paiements à des tiers agréés, sous consentement du client.

> **Open Banking** — l'ouverture effective des données de compte et des capacités de paiement à des tiers agréés, avec le consentement explicite du client. La DSP2 est la règle, l'Open Banking la pratique, et l'**API** en est l'outil technique — celui que l'on étudie aujourd'hui.

Le fil conducteur du Jour 2

Une API Open Banking protège trois choses en même temps : l'**identité du tiers**, l'**autorisation du client** et l'**intégrité de chaque appel**. Chaque bloc du jour correspond à l'une de ces protections.

APERÇU

Le parcours du Jour 2, en une image

Trois temps, du composant au cas concret. On part de la brique API, on monte jusqu'aux mécanismes d'authentification, puis aux exigences de sécurité — et le cas pratique S7 rejoue l'ensemble de bout en bout.

S1 — Architecture API

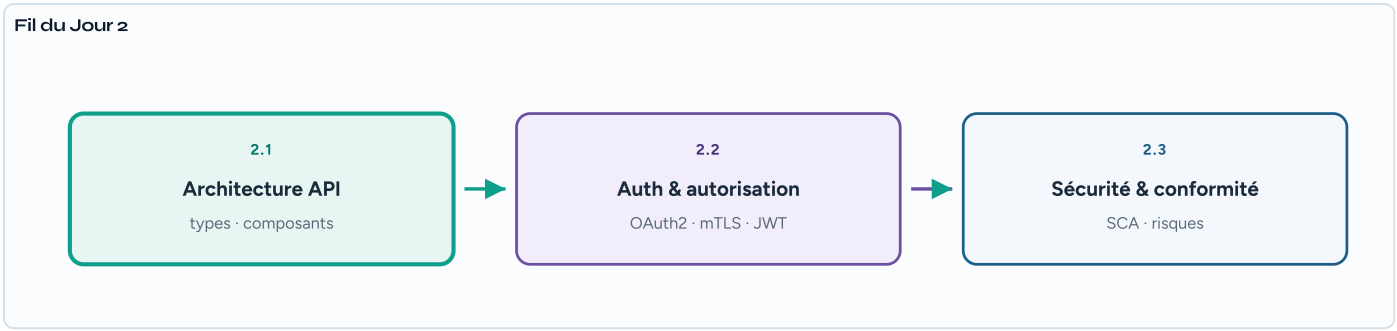
Les types d'API (AIS, PIS, CBPII) et les composants qui les exposent : Gateway, API Management, throttling, logs.

S2 — Authentification & autorisation

OAuth2, OpenID Connect, mTLS, JWT : qui prouve son identité, et comment l'accès aux données est-il borné ?

S3 — Sécurité & conformité

Exigences DSP2 : SCA, exemptions, non-répudiation, horodatage. Et les risques : usurpation, attaques API, fuites.



OBJECTIFS PÉDAGOGIQUES

À l'issue du Jour 2, l'étudiant sait…

Distinguer les trois types d'API Open Banking — Account Information (AIS), Payment Initiation (PIS), Confirmation of Funds (CBPII) — et leurs composants (Gateway, API Management, throttling).

Expliquer les mécanismes d'authentification et d'autorisation : OAuth2, OpenID Connect, mTLS, JWT, et la séparation entre identité du client, identité du TPP et autorisation d'accès.

Relier les exigences DSP2 de sécurité (SCA, exemptions, non-répudiation, horodatage) aux mécanismes techniques qui les mettent en œuvre.

Identifier les risques propres aux API (usurpation d'identité, attaques API, fuites de données) et les contre-mesures associées.

Concevoir l'ouverture sécurisée d'une API bancaire à un TPP : schéma d'architecture, flux OAuth et points de contrôle de sécurité.

FIL DE SÉANCE

Parcours

CURSUS

Trajectoire des 5 jours



S1 - Architecture API

UNE INFRASTRUCTURE, TROIS SERVICES

Architecture API — le socle commun

AIS, PIS, CBPII : trois API aux usages différents, mais bâties sur **la même infrastructure d'exposition**. On commence par ce socle commun — passerelle, gestion, services, journalisation — avant de détailler chaque API dans les séquences suivantes.

FIL ROUGE

L'application de Lyam n'utilise pas « une API » : elle appelle une API de **lecture de comptes** (AIS). Demain, si Lyam paie un marchand par virement, ce sera une API d'**initiation de paiement** (PIS). Derrière ces appels différents, la banque expose toujours la *même* plomberie : c'est ce socle qu'on découvre ici.

ORIENTATION

Trois API, un même socle

La DSP2 définit trois familles de services, chacune exposée par une API dédiée. Leur *logique métier* diffère ; leur *infrastructure d'exposition* est commune. Cette séquence pose le socle ; les deux suivantes détaillent le workflow propre à l'AIS et au CBPII. Le PIS, lui, fait l'objet du Jour 3.

Account Information API

LECTURE DES COMPTES · AIS

Lit soldes, opérations et détails de comptes. **Lecture seule**, aucun mouvement de fonds.

→ S2 · détail de l'API AIS

Payment Initiation API

INITIATION DE PAIEMENT · PIS

Déclenche un virement au nom du client, sans jamais détenir les fonds.

→ détaillé au Jour 3

Confirmation of Funds API

VÉRIFICATION DE FONDS · CBPII

Répond « fonds disponibles ? » sans exposer de solde.

→ S3 · workflow CBPII

Une API, un usage

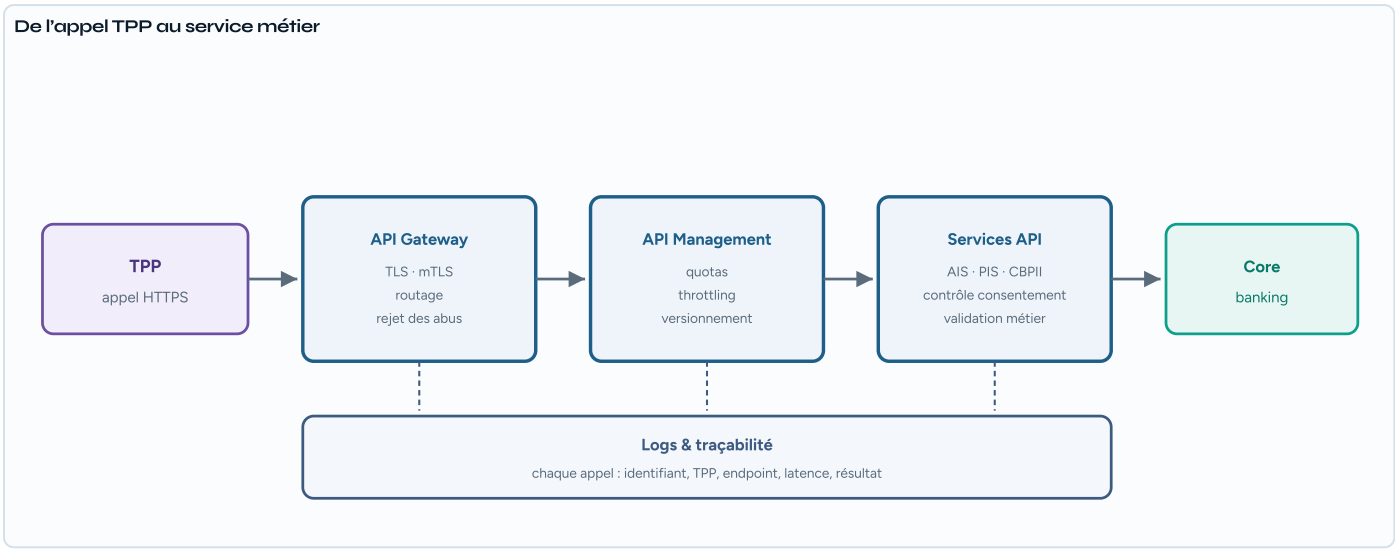
Une banque peut exposer les trois types, mais un TPP n'accède qu'aux types couverts par son agrément. Un AISP n'obtient jamais le droit d'appeler l'API de paiement : la séparation est technique *et* réglementaire.

COMPOSANTS

Les composants qui exposent l'API

Quelle que soit l'API appelée, l'appel traverse la même chaîne de composants. On retrouve ici les couches du

Jour 1, vues cette fois du côté de l'exposition.



C'est ici que tout le module s'appuie : prenons le temps de fixer chaque composant, un par un. Pour chacun : **ce que c'est**, **son rôle technique**, et une **image mentale** pour le retenir.

1

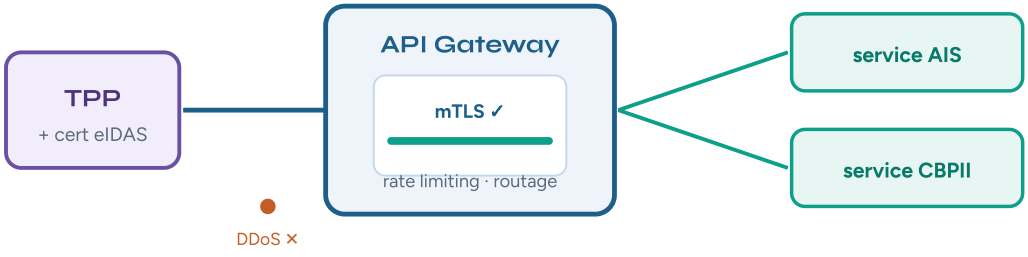
API Gateway

La porte d'entrée · le gardien

Ce que c'est
Le **premier composant interne** que le TPP touche en arrivant sur le réseau de la banque. C'est un serveur logiciel qui sert de **point d'entrée unique** (un *reverse proxy*) : aucun appel n'atteint les services internes sans passer par lui.

Son rôle technique

- **Faire respecter le mTLS** — chiffrement et vérification mutuelle des certificats eIDAS du TPP.
- **Filtrer le trafic** — rejeter les appels malformés ou non conformes avant qu'ils n'atteignent le cœur.
- **Limiter le débit** (*rate limiting*) — borner le nombre de requêtes par seconde pour parer aux attaques par déni de service (DDoS).
- **Router la requête** vers le bon service interne, selon l'endpoint appelé.



Le TPP présente son certificat ; la gateway vérifie (mTLS), filtre le trafic abusif et route vers le bon service.

Image mentale
Le **vigile à l'entrée** de la banque : il contrôle l'identité, refuse les indésirables et indique le bon guichet.

2

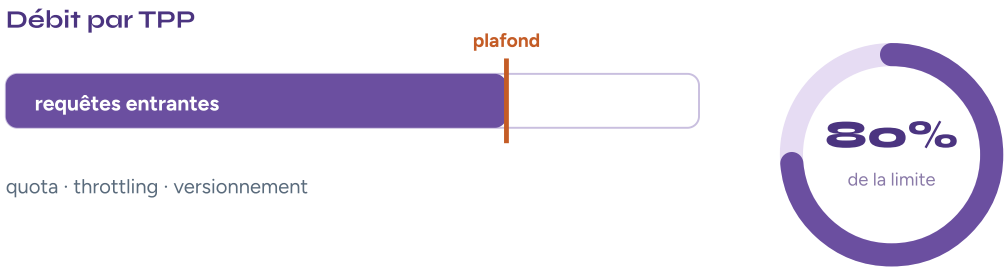
API Management

Le bureau des règles · le régulateur

Ce que c'est
La couche qui **gouverne** l'usage des API : le catalogue des API exposées et, pour chaque TPP, les droits et les limites. Là où la Gateway laisse entrer, le Management décide *combien* et *à quelle fréquence*.

Son rôle technique

- **Quotas par TPP** — nombre d'appels autorisés sur une période.
- **Throttling** — limitation de débit pour lisser la charge et protéger la disponibilité.
- **Versionnement** — gérer plusieurs versions d'une même API sans casser les TPP existants.
- **Catalogue & politiques** — qui a le droit d'appeler quoi, et sous quelles conditions.



> Chaque TPP dispose d'un quota ; au-delà du plafond, les appels sont limités (throttling) pour préserver la disponibilité.

Image mentale
Le **régulateur de vitesse** : il fixe la limite autorisée et empêche qu'un seul usager sature la route pour tous.
3

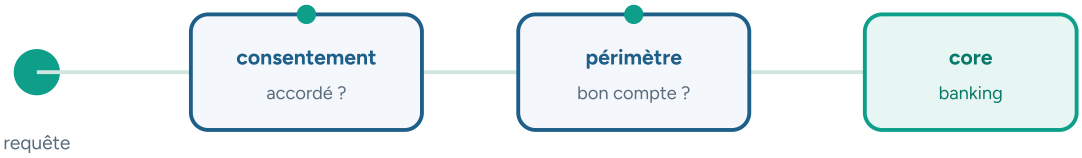
Services API métier

Le cœur du métier · le spécialiste

Ce que c'est
Les services qui **implémentent réellement** AIS, PIS et CBPII. C'est la couche où chaque API devient *elle-même* : après le contrôle d'entrée, c'est ici que se joue le comportement propre à chaque service.

Son rôle technique

- **Vérifier le consentement** — l'accès demandé est-il couvert ?
- **Contrôler le périmètre** — bons comptes, bonnes opérations, bonne durée.
- **Valider les règles métier** avant d'appeler le core banking.
- **Spécialisation** — workflow AIS (S2), CBPII (S3), PIS (Jour 3).



> Avant tout accès au core, le service métier vérifie le consentement et le périmètre : seuls les bons comptes, pour les bonnes opérations.

Image mentale
Le **guichetier spécialiste** : c'est lui qui sait si vous avez le droit d'accéder à ce compte, et qui traite la demande qui vous concerne.

4

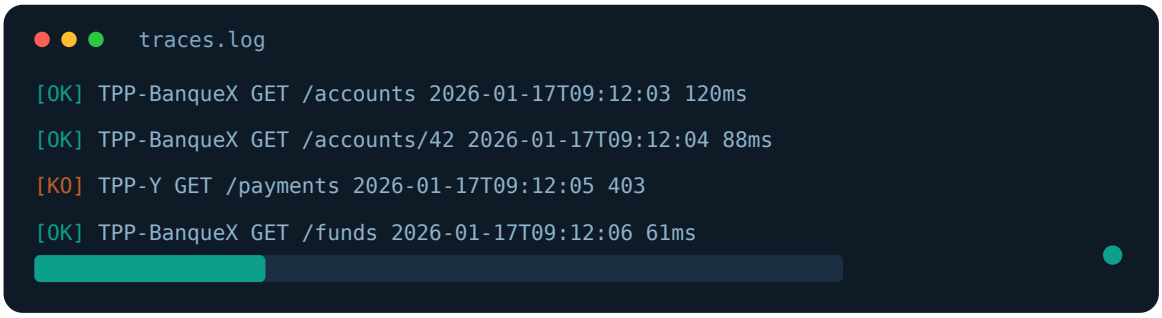
Logs & traçabilité

La mémoire de la banque · le greffier

Ce que c'est
La couche qui **journalise chaque appel** de bout en bout. Elle ne bloque rien : elle *garde la preuve*. Indispensable à l'audit, aux litiges et au monitoring.

Son rôle technique

- **Qui** — quel TPP a appelé.
- **Quoi** — quel endpoint, quelles données demandées.
- **Quand** — horodatage de l'appel (non-répudiation).
- **Résultat** — code de retour, latence, incident.



Qui (TPP), quoi (endpoint), quand (horodatage), résultat (code + latence) : la mémoire auditable de chaque appel.

Image mentale
Le **greffier** : il consigne chaque visite, qui est entré, pour quoi faire et ce qui s'est passé — utile le jour où l'on doit reconstituer les faits.

Throttling : pourquoi c'est une exigence, pas une option

Les RTS imposent des plafonds de consultation. Sans limitation de débit, un TPP mal codé peut saturer l'API et faire tomber, pour tous les autres, l'accès aux comptes. Le throttling protège la disponibilité du service — donc la qualité pour Lyam.

SÉQUENCE SUIVANTE

Maintenant, chaque API en détail

Le socle est posé. On peut désormais spécialiser : chaque API a son cœur métier, ses endpoints, ses règles SCA et ses risques propres. Commençons par la plus utilisée : la lecture de comptes (S2 · AIS).

03 · S2 · WORKFLOW AIS

S2 · Workflow AIS

LIRE LES COMPTES, JAMAIS LES TOUCHER

Workflow AIS — Account Information

L'API que Lyam utilise au quotidien : lire ses soldes et ses opérations depuis son application. Sur le socle de S1, elle ajoute un cœur métier précis : **lecture seule**, cadrée par un consentement et une authentification forte initiale.

FIL ROUGE

Lyam ouvre son application de gestion de budget. Elle affiche la liste de ses comptes, le solde de chacun et ses dernières opérations. Pour cela, elle appelle l'API AIS de Banque Est avec le consentement que Lyam a donné une fois — et qui reste valable jusqu'à 90 jours.

CŒUR MÉTIER

Ce que fait — et ne fait pas — l'API AIS

Ce qu'elle fait

- Lister les comptes (comptes courants, livrets, cartes)
- Donner le détail d'un compte (IBAN, titulaire, type)
- Lister les transactions d'un compte, filtrables par date
- Donner les soldes (comptabilisé, disponible)

Ce qu'elle ne fait jamais

- Aucun mouvement de fonds (pas de virement, pas de prélèvement)
- Aucune modification du compte
- Aucun accès sans consentement AIS actif
- Aucun accès aux comptes non couverts par le consentement

LE PRINCIPE

L'AISP lit ; il ne touche à rien. Le risque de l'AIS n'est pas la fraude financière, c'est la *confidentialité* des données de comptes.

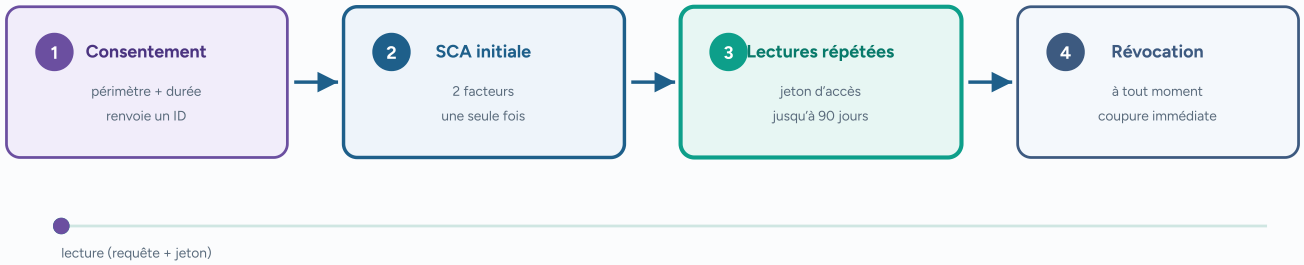
PARCOURS

Le parcours d'une lecture de comptes

Une lecture AIS suit toujours la même séquence : consentement, authentification forte initiale, puis lecture

répétée tant que le consentement est valide.

Flux d'une lecture AIS



1

Consentement

Le badge d'accès

Ce qui se passe
L'AISP déclare **ce qu'il veut lire** (comptes, transactions, soldes) et **pour combien de temps**. La banque renvoie un identifiant de consentement.

Image mentale
Un **badge nominatif** : il précise les portes ouvertes et la date d'expiration.

2

SCA initiale

La clé jetable

Ce qui se passe
Lyam s'authentifie fortement (deux facteurs) et autorise explicitement le consentement. C'est le **seul moment** où il prouve son identité.

Image mentale
Une **serrure à deux clés** qu'on tourne une seule fois ; ensuite la porte reste entrouverte.

3

Lectures répétées

La fenêtre de lecture

Ce qui se passe
Avec un jeton d'accès, l'AISP lit comptes et transactions **sans nouvelle SCA**, tant que le consentement est valide (**90 jours max** sous RTS).

Image mentale
Une **fenêtre ouverte 90 jours** : on peut regarder dehors autant de fois qu'on veut, sans rouvrir.

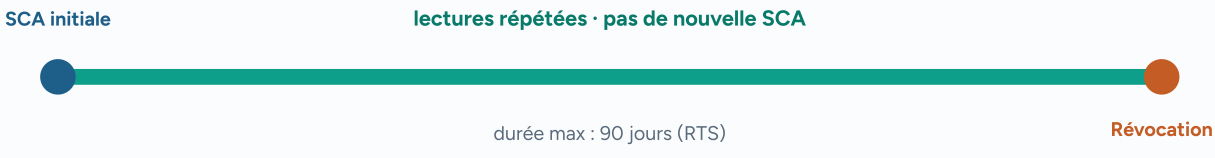
4

Révocation

L'interrupteur

Ce qui se passe
Lyam révoque le consentement **à tout moment**. L'accès est coupé immédiatement, quels que soient les jetons encore en circulation.

Image mentale
Un **interrupteur général** : on coupe, tout s'éteint, aucune copie ne survit.



Une seule authentification forte au départ ; ensuite, une fenêtre de lecture qui se referme au plus tard à 90 jours — ou dès que Lyam révoque.

Figure — le parcours AIS : une seule authentification forte initiale, puis des lectures répétées tant que le consentement reste valide (90 jours max).

ÉTAPE	ENDPOINT TYPIQUE (BERLIN GROUP)	BUT
Consentement	`POST /v1/consents`	Déclarer le périmètre et la durée
Comptes	`GET /v1/accounts`	Lister les comptes autorisés
Détail	`GET /v1/accounts/{accountId}`	IBAN, titulaire, type
Soldes	`GET /v1/accounts/{accountId}/balances`	Solde comptabilisé / disponible
Transactions	**`GET /v1/accounts/{accountId}/transactions`**	**Opérations, filtrables par date**

Les noms exacts varient selon le standard (Berlin Group, STET, OBIE). Ce qui est normalisé par la DSP2, c'est le service rendu, pas l'URL.

DONNÉES

À quoi ressemble une réponse AIS

L'AIS renvoie des données riches — c'est justement ce qui le rend sensible. On lit les transactions d'un compte par un simple **GET** authentifié :

TYPE D'API	MÉTHODE	AUTH	CONSENTEMENT
Account Information (AIS)	GET	Bearer (OAuth2)	AIS actif requis

GET **/accounts/{id}/transactions** — REQUÊTE

```
GET /accounts/FR76…1234/transactions
Authorization: Bearer eyJhbGciOi...
Consent-ID: cons-9f2a-ais

# paramètres optionnels
?dateFrom=2025-11-01&dateTo=2025-11-30
```

200 **application/json** — RÉPONSE

```
{
  "account": { "iban": "FR76…1234" },
  "transactions": {
    "booked": [
      {
        "transactionId": "TX-2025-0001",
        "bookingDate": "2025-11-08",
        "amount": { "currency": "EUR", "amount": "-42.90" },
        "creditorName": "SNCF",
        "remittanceInformation": "Billet Paris–Lyon"
      },
      {
        "transactionId": "TX-2025-0002",
        "bookingDate": "2025-11-07",
        "amount": { "currency": "EUR", "amount": "2500.00" },
        "debtorName": "ACME SARL",
        "remittanceInformation": "Salaire novembre"
      }
    ]
  }
}
```

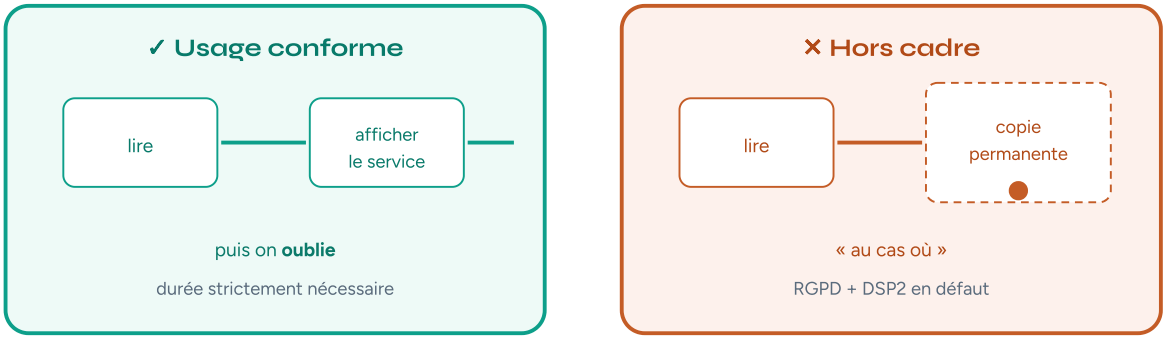
Pourquoi c'est sensible

Une réponse AIS révèle les revenus, les habitudes d'achat, les bénéficiaires, la santé financière de Lyam. C'est une donnée personnelle au sens du RGPD : d'où le consentement explicite, la durée bornée et le droit de révocation à tout moment.

RISQUES

Les erreurs et risques propres à l'AIS

CODE HTTP	SIGNIFICATION	CE QUE FAIT L'AISP
401	Jeton absent ou invalide	Re-authentifier, redemander la SCA si besoin
403	Consentement expiré ou révoqué	Demander un nouveau consentement à Lyam
429	Trop de requêtes (throttling)	Ralentir, respecter les plafonds RTS
503	**Service indisponible**	**Réessayer avec backoff, prévenir l'utilisateur**



Lire pour rendre le service et oublier ensuite — ou constituer un entrepôt de données permanent : la frontière DSP2/RGPD se joue là.

Le piège : le stockage des données

L'AISP ne doit conserver les données lues que pour la durée strictement nécessaire au service rendu. Un AISP qui garde une copie intégrale des transactions « au cas où » sort de son cadre — et s'expose au RGPD comme à la DSP2.

SÉQUENCE SUIVANTE

Et si l'on ne faisait que vérifier ?

L'AIS lit beaucoup. La troisième API, elle, répond à une seule question — « y a-t-il assez de fonds ? » — sans révéler de solde. C'est un tout autre workflow, minimaliste par choix (S3 · CBPII).

04 · S3 · WORKFLOW CBPII

S3 - Workflow CBPII

UNE SEULE QUESTION, UNE SEULE RÉPONSE

Workflow CBPII — Confirmation of Funds

La plus discrète des trois API : elle répond à une unique question — « y a-t-il assez de fonds pour ce montant ? » —

par un simple **oui ou non**, sans jamais révéler de solde ni d'opération.

FIL ROUGE

Lyam a une carte de paiement adossée à un porte-monnaie géré par un émetteur. Avant d'autoriser une dépense, l'émetteur demande à Banque Est : « le compte de Lyam couvre-t-il 42,90 € ? ». Réponse : oui ou non. L'émetteur n'apprend ni le solde, ni les opérations de Lyam.

CŒUR MÉTIER

Une API volontairement pauvre

Là où l'AIS expose des données riches, le CBPII expose le minimum nécessaire. C'est un choix de conception protecteur : moins de données exposées, moins de risque.

Ce qu'elle fait

- Répond oui/non à « fonds suffisants pour tel montant ? »
- Se limite au compte et au montant demandés
- Sert de contrôle avant une autorisation de paiement

Ce qu'elle ne fait jamais

- Renvoyer un solde (même approximatif)
- Lister des transactions
- Déplacer des fonds
- Donner des informations sur d'autres comptes

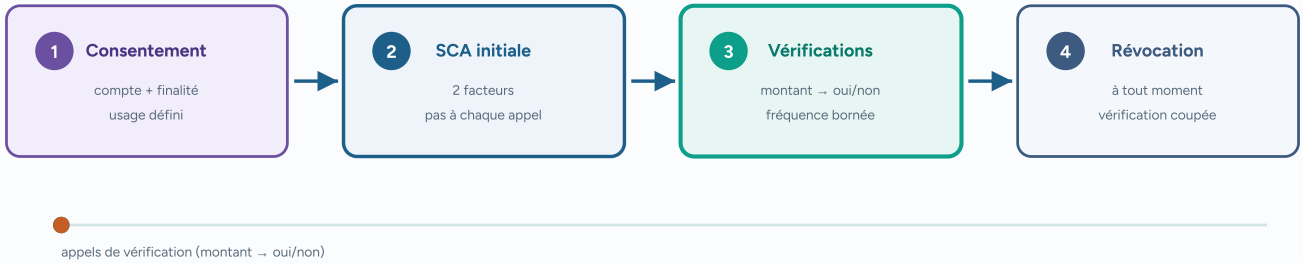
LE PRINCIPE

Le CBPII ne lit pas : il *vérifie*. Une réponse binaire vaut mieux qu'un solde exposé — c'est la protection par la pauvreté de la donnée.

PARCOURS

Le parcours d’une vérification de fonds

Flux d’une vérification CBPII



1

Consentement

Le badge d’accès

Ce qui se passe
Le CBPII déclare **le compte à vérifier** et **la finalité du contrôle**. La banque renvoie un identifiant de consentement.

Image mentale
Un **badge nominatif** : il précise le compte concerné et la raison du contrôle.

2

SCA initiale

La clé jetable

Ce qui se passe
Lyam s’authentifie fortement (deux facteurs) et autorise explicitement le consentement. C’est le **seul moment** où il prouve son identité.

Image mentale
Une **serrure à deux clés** qu’on tourne une seule fois ; ensuite la porte reste entrouverte.

3

Vérifications

Le voyant lumineux

Ce qui se passe
Pour chaque dépense, le CBPII envoie le montant et reçoit un booléen. Pas de nouvelle SCA, **jusqu’à 90 jours** (RTS).

Image mentale
Un **voyant lumineux** qui répond oui/non : « suffisant ? »

4

Révocation

L’interrupteur

Ce qui se passe
Lyam révoque le consentement **à tout moment**. Le CBPII perd alors toute possibilité de vérifier les fonds.

Image mentale
Un **interrupteur général** : on coupe, tout s’éteint.

Figure — le parcours CBPII : même squelette que l’AIS (consentement, SCA initiale, révocation), mais la boucle répétée ne renvoie qu’un booléen « fonds suffisants ».



Un montant envoyé, un booléen reçu — sans aucun autre détail.

ÉTAPE	ENDPOINT TYPIQUE (BERLIN GROUP)	BUT
Consentement	`POST /v1/funds-confirmations`	Déclarer le compte à vérifier
Vérification	appel avec montant + compte	Obtenir « fonds suffisants : oui/non »

DONNÉES

À quoi ressemble une réponse CBPII

La réponse est minimale. Pas de montant de solde, pas de transaction : un seul indicateur booléen. La question posée est un simple **POST** authentifié :

TYPE D'API	MÉTHODE	AUTH	CONSENTEMENT
Confirmation of Funds (CBPII)	POST	Bearer (OAuth2)	PIIS actif requis

POST `/funds-confirmations` — REQUÊTE

```
POST /funds-confirmations
Authorization: Bearer eyJhbGciOi...
Consent-ID: cons-c4d1-piis

{
  "account": { "iban": "FR76…1234" },
  "instructedAmount": {
    "currency": "EUR",
    "amount": "120.00"
  }
}
```

200 `application/json` — RÉPONSE

```
{
  "fundsAvailable": true,
  "account": { "iban": "FR76…1234" }
}
```



Le CBPII ne renvoie jamais le solde — seulement une réponse binaire.

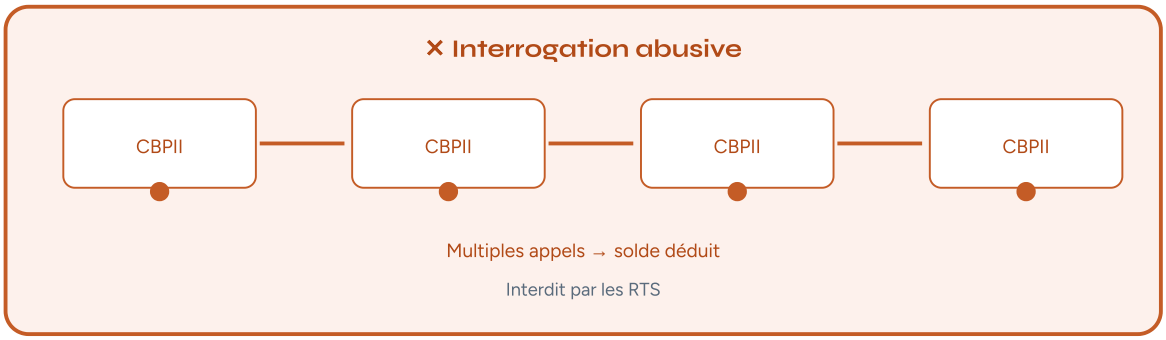
Un booléen, pas un solde

Pourquoi ne pas simplement renvoyer le solde ? Parce que le solde est une donnée sensible. Le CBPII n'a besoin que de la réponse à sa question : le minimum exposé est le maximum protégé.

USAGES

Cas d'usage et limites

CAS D'USAGE	INTÉRÊT DU CBPII
Porte-monnaie / carte prépayée	Vérifier la couverture avant d'autoriser
Émetteur de carte	Contrôle de solvabilité sans voir les comptes
Paiement en plusieurs fois	**S'assurer de la capacité de remboursement**



Multiplier les appels pour déduire le solde est interdit — les RTS limitent la fréquence.

Le piège : l'interrogation abusive

Multiplier les vérifications permet de reconstituer un solde par déduction. Les RTS limitent donc la fréquence d'interrogation — c'est le même enjeu de *throttling* vu en S1, mais ici pour protéger une donnée déduite, pas un service.

05 · S5 · AUTHENTIFICATION & AUTORISATION

S5 - Authentification & autorisation

QUI PROUVE QUOI, ET JUSQU'OÙ

Authentification & autorisation

Quatre mécanismes reviennent sans cesse en Open Banking : OAuth2, OpenID Connect, mTLS et JWT. Chacun répond à une question différente. Les confondre, c'est ouvrir une faille.

FIL ROUGE

Quand l'application de Lyam appelle Banque Est, trois choses doivent être vraies : **l'appel vient d'un TPP agréé**, **Lyam s'est authentifiée**, et **l'accès demandé est bien couvert par son consentement**. Un seul mécanisme ne suffit pas.

MÉCANISMES

Les quatre mécanismes clés

Attention au vocabulaire : **authentifier**, c'est prouver *qui* est quelqu'un ; **autoriser**, c'est décider *à quoi* il a droit. Les quatre mécanismes ci-dessous ne font pas la même chose : deux servent plutôt l'authentification, deux

servent plutôt l'autorisation.

Le sigle à connaître : le SCA

SCA = Strong Customer Authentication (*authentification forte du client*) : le mécanisme par lequel **Lyam prouve son identité** auprès de sa banque, en combinant **au moins deux facteurs** de catégories différentes (connaissance / possession / inhérence). Côté technique, il s'appuie souvent sur **OpenID Connect** pour produire un ID token signé.

Le détail du SCA (facteurs, moments d'application, exemptions) est traité dans la séquence suivante, S6 — Sécurité & conformité. Ici, on retient seulement qu'il alimente la preuve d'identité du client.

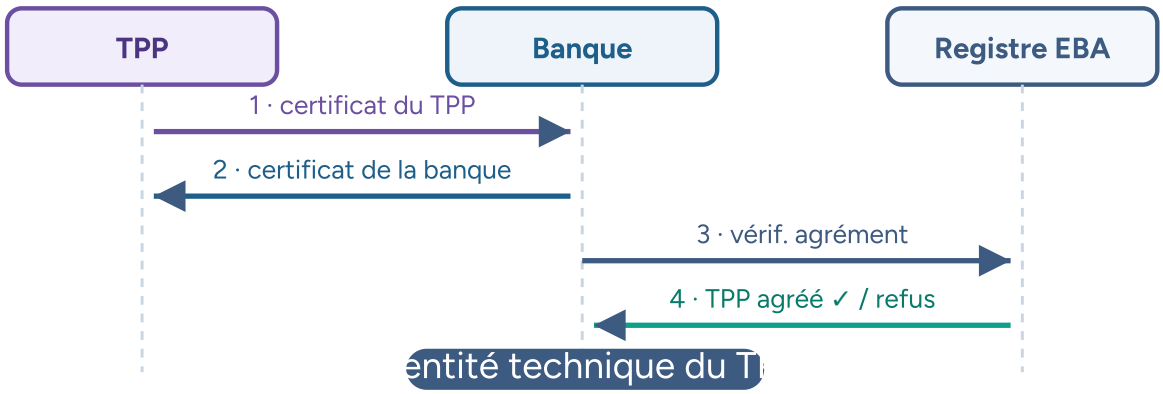
NOTION	QUESTION POSÉE	RÉPONSE ATTENDUE	MÉCANISMES
Authentification	Qui es-tu ?	Prouve ton identité	mTLS, OpenID Connect
Autorisation	**À quoi as-tu droit ?**	**Voici ton périmètre et tes jetons**	**OAuth2 (+ scopes du JWT)**

1

mTLS

Mutual TLS · la poignée de main et le canal sécurisé

Le point de départ
Tout commence par la **poignée de main** : avant même de parler d'autorisation, le canal doit être sûr et l'identité technique du TPP prouvée.



Le canal est établi seulement si les deux certificats sont valides : mTLS prouve que le TPP est le TPP agréé.

1mTLS (Mutual Transport Layer Security) — La poignée de main et le canal sécurisé

Ce que c'est

- Une extension du protocole TLS (le HTTPS classique).
- **Les deux parties s'authentifient mutuellement** grâce à des certificats numériques.
- Le HTTPS ordinaire n'authentifie que le serveur ; mTLS authentifie *aussi* le client.

Le problème qu'il résout

- Sur le web classique, seul le serveur (la banque) prouve son identité au client.
- Dans la DSP2, la banque doit **aussi vérifier le TPP** de façon instantanée et cryptographique.
- Objectif : n'ouvrir la porte qu'à un client dont l'identité technique est prouvée.

Comment ça marche en DSP2 — la poignée de main mTLS

1. Le TPP initialise une requête de connexion **HTTPS** vers l'API Gateway de la banque.
2. La **poignée de main (handshake) mutuelle** s'engage :
    - La banque présente son certificat TLS pour prouver qu'elle est bien l'établissement légitime.

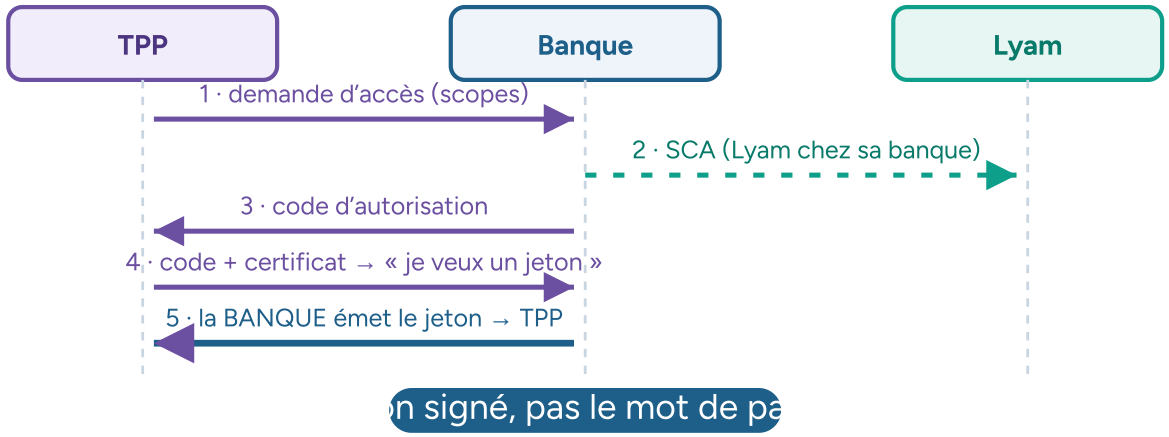
    - En retour, le TPP présente obligatoirement son **certificat eIDAS (QWAC)** délivré par une autorité de confiance qualifiée.
3. L'API Gateway de la banque intercepte ce certificat et valide à la volée qu'il est **en cours de validité**, qu'il **n'est pas révoqué**, et qu'il appartient bien à un **TPP officiellement agréé** (par l'ACPR ou une autre autorité européenne).
4. Si et seulement si cette « poignée de main » cryptographique réussit, le **tunnel sécurisé HTTPS** est établi.

À retenir : le mTLS sécurise le *tuyau* (la couche transport) entre le TPP et la banque. Sans certificat eIDAS valide lors de cette poignée de main, la connexion est immédiatement rejetée.
2

OAuth2

Authorization Framework · autorisation

Ce que c'est
Cadre d'**autorisation**. Il permet à un TPP d'obtenir un **jeton d'accès** sans jamais connaître les identifiants de Lyam. Il ne dit *pas* qui est Lyam : il dit *à quoi* le TPP a droit.



Le **jeton n'est pas créé par le TPP** : c'est la **banque (ASPSP)** qui le forge et le signe (étape 5), puis le renvoie au TPP en réponse à l'échange du code. Le TPP repart avec un titre d'accès signé, jamais avec les identifiants de Lyam.

2OAuth2 — Le cadre d'autorisation qui délivre les jetons

Ce que c'est

- **OAuth2 = Open Authorization, version 2** : un standard ouvert de *délégation d'autorisation* (RFC 6749).
- Un cadre d'**autorisation** : il dit *à quoi* un TPP a droit, pas *qui* est l'utilisateur.
- Il décrit comment un TPP obtient, avec l'accord de Lyam, un **jeton d'accès** à périmètre limité.
- Lyam ne confie *jamais* ses identifiants bancaires au TPP.

Le problème qu'il résout

- Comment donner à un tiers un accès *partiel et révocable* à ses comptes, sans lui livrer son mot de passe ?
- OAuth2 sépare « prouver qui je suis » (fait par la banque lors de la SCA) de « ce que le TPP peut faire » (le jeton et ses *scopes*).

Comment ça marche en DSP2 — le flux code d'autorisation

1. Le TPP demande l'accès en précisant les **scopes** (quels comptes, quelles opérations).
2. Lyam réalise la **SCA** chez sa banque et accorde son consentement.
3. La banque renvoie au TPP un **code d'autorisation** à usage unique.
4. Le TPP échange ce code (sur le canal mTLS) contre un **access token** — et un **refresh token** pour le renouveler.

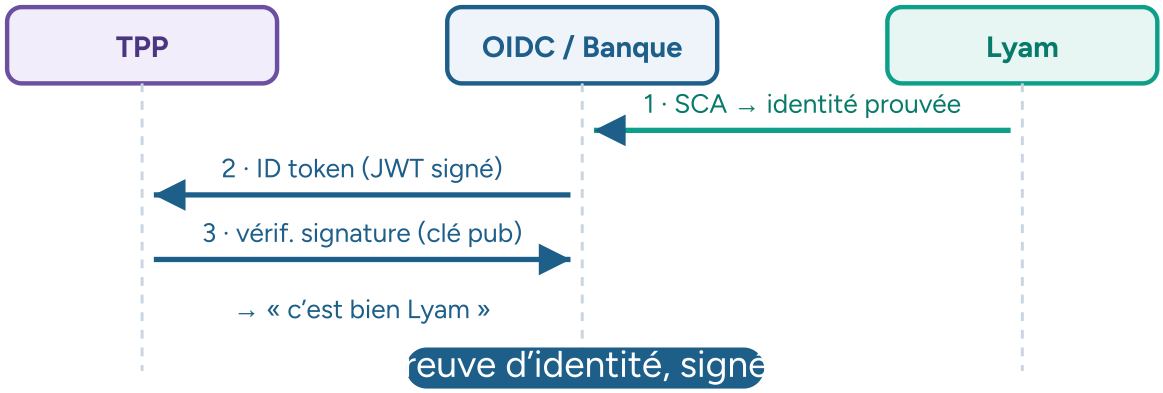
À retenir : OAuth2 n'authentifie pas Lyam, il *autorise* le TPP. Le jeton porte le périmètre : hors des scopes accordés, l'appel est refusé (403).

3

OpenID Connect

Couche d'identité sur OAuth2 · authentification

Ce que c'est
Ajoute l'**identité** à OAuth2 : un **ID token** (JWT) qui atteste qui est l'utilisateur authentifié. C'est ce qui transforme un simple « droit d'accès » en « accès pour *cette* personne ».



OIDC ajoute à OAuth2 un ID token : le TPP sait *qui* est l'utilisateur, pas seulement ce à quoi il a droit.

3OpenID Connect — L'identité par-dessus OAuth2

Ce que c'est

- Une couche d'**identité** construite sur OAuth2.
- Elle ajoute un **ID token** — un JWT signé — qui atteste *qui* est l'utilisateur.
- Le jeton est vérifiable cryptographiquement : on ne croit pas l'émetteur sur parole.

Le problème qu'il résout

- OAuth2 seul autorise un accès mais ne dit pas *à qui* il appartient.
- Pour savoir que c'est bien Lyam (et pas « un utilisateur quelconque »), il faut une preuve d'identité.
- C'est précisément le rôle d'OIDC.

Comment ça marche en DSP2

1. Après la **SCA** chez la banque, celle-ci génère un **ID token** signé contenant l'identité de Lyam.
2. Le TPP reçoit l'ID token et **vérifie sa signature** avec la clé publique de l'émetteur : « c'est bien Lyam ».
3. Pendant que **l'access token OAuth2** porte les droits, **l'ID token OIDC** porte l'identité : deux jetons, deux rôles.

À retenir : OAuth2 = « à quoi as-tu droit ? ». OIDC = « qui es-tu ? ». On les combine : l'un autorise, l'autre authentifie.

4

JWT

JSON Web Token · le support des preuves

Ce que c'est
Format de jeton **signé**. Il transporte des affirmations (scopes, expiration, émetteur) vérifiables sans rappeler le serveur à chaque fois. C'est le *contenant* : selon son contenu, il sert l'authentification (ID token) ou l'autorisation (access token avec scopes).



Si la signature est valide → le jeton est intact →
on lit les scopes, on contrôle `exp`, on fait confiance à l'identité

⚠ lisible par tous (non chiffré) : ne jamais y mettre de données sensibles

JWT = `header.payload.signature`. **On lit** les affirmations du payload ; **on vérifie** la signature avec la clé publique de l'émetteur. Lisible ≠ chiffré : la signature garantit l'intégrité, pas la confidentialité.

4JWT — L'enveloppe signée qui transporte la preuve

Ce que c'est

- Un **JSON Web Token**, une chaîne `header.payload.signature`.
- Un en-tête, une charge utile d'affirmations (scopes, expiration, émetteur) et une **signature** cryptographique.

Le problème qu'il résout

- Transporter une preuve (identité ou droit) de façon compacte.
- Permettre aux services de lire la preuve *sans* re-interroger le serveur à chaque appel.
- Garantir que la preuve **n'a pas été altérée** en route.

Comment ça marche en DSP2

- Le **payload** est encodé mais *lisible* par tous : on ne doit jamais y mettre de données sensibles.
- La **signature** est vérifiée avec la clé publique de l'émetteur : si elle est valide, le jeton est intact.
- Le champ **exp** impose une date d'expiration : un jeton périmé est refusé.

À retenir : le JWT n'est ni « authentification » ni « autorisation » : c'est le *support signé* qui transporte l'une ou l'autre preuve (ID token ou access token).

MÉCANISME	RÉPOND À LA QUESTION	NIVEAU
mTLS	Le TPP est-il bien celui qu'il prétend ?	Canal — authentification
OAuth2	À quoi le TPP a-t-il droit ?	Autorisation
OpenID Connect	Qui est l'utilisateur authentifié ?	Identité — authentification
JWT	**Comment transporter la preuve, signée ?**	**Jeton — support**

LE POINT CLÉ

Trois vérifications distinctes, à ne jamais confondre

Une requête Open Banking valide repose sur **trois contrôles indépendants**. Ils concernent trois acteurs différents (le TPP, Lyam, l'accès aux données), se font à des endroits différents de la chaîne et reposent sur des

mécanismes différents. Fusionner les trois, c'est ouvrir une faille.

CONTRÔLE	QUI PROUVE QUOI ?	MÉCANISME
1. Identification du TPP	L'**application** prouve qu'elle est un TPP agréé (identité technique)	mTLS + certificat eIDAS (registre EBA)
2. Identification de Lyam	Le **client** prouve qui il est auprès de sa banque	SCA / OpenID Connect
3. Autorisation d'accès	**Le périmètre concédé (comptes, opérations, durée) est valide**	**OAuth2 + consentement (scopes du JWT)**

1 · Identification du client

Lyam prouve qui il est auprès de sa banque (SCA). C'est lui, physiquement et juridiquement, qui est à l'origine de la demande. *Mécanisme : SCA / OIDC.*

2 · Identification du TPP

L'application prouve qu'elle est un TPP agréé (certificat, agrément). C'est une identité technique, liée au registre EBA. *Mécanisme : mTLS.*

3 · Autorisation d'accès

Le périmètre concédé à ce client et à ce TPP : quels comptes, quelles opérations, jusqu'à quand. C'est le consentement, matérialisé en jetons. *Mécanisme : OAuth2.*

Les trois sont nécessaires — en manquer une = faille

- **TPP agréé mais pas de consentement** → un accès sans accord du client : interdit.
- **Client authentifié mais TPP non agréé** → un appelant non fiable dans le système : interdit.
- **Consentement trop large exploité par un TPP malveillant** → abus de périmètre : interdit.

Chaque contrôle se fait **indépendamment** : réussir l'un ne dispense jamais des deux autres.

RÈGLE À RETENIR

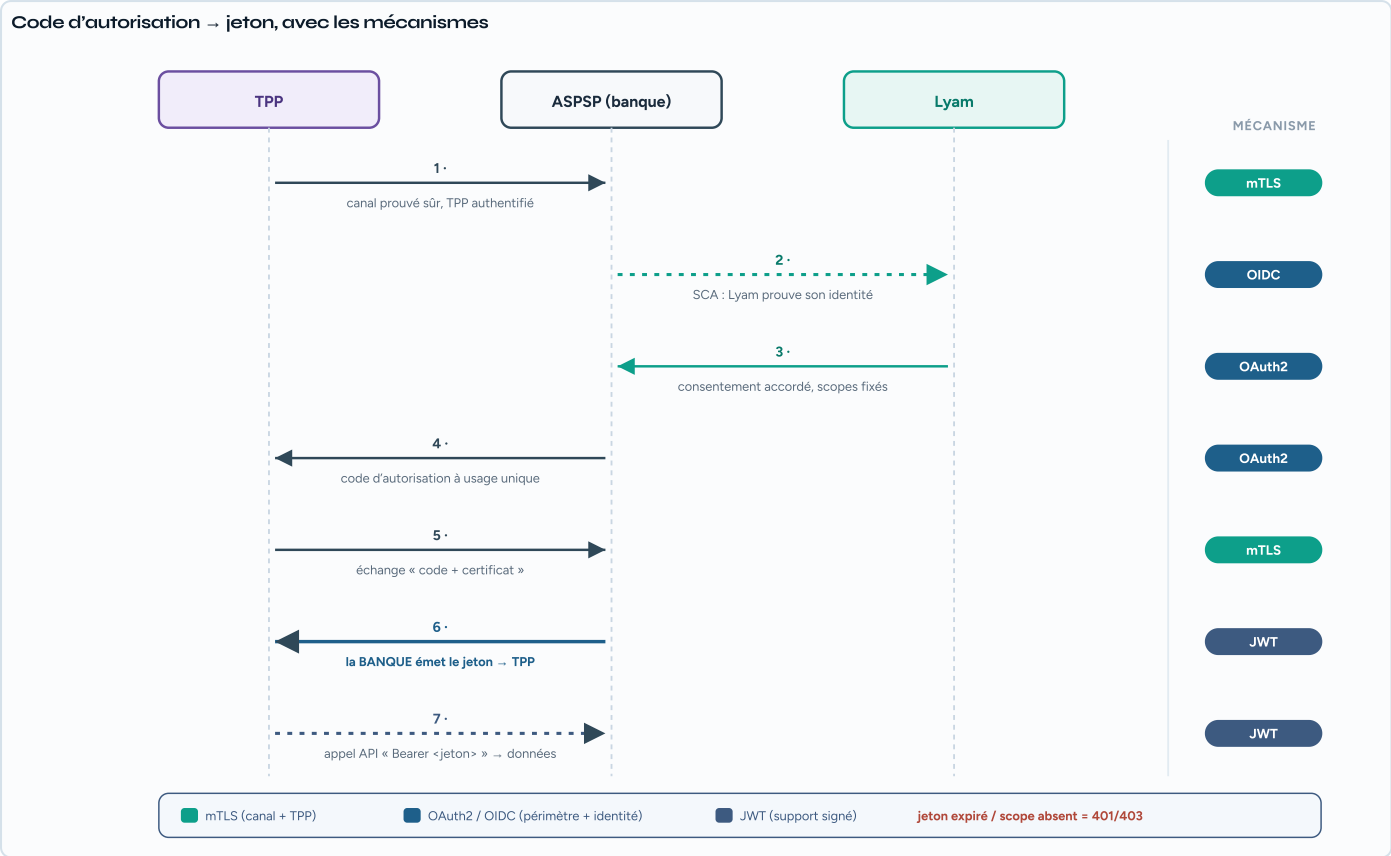
Authentifier le TPP (mTLS) ≠ **authentifier le client** (SCA/OIDC) ≠ **autoriser l'accès aux données** (OAuth2). Trois questions, trois moments, trois mécanismes : on ne peut pas en sauter un.

FLUX

Le flux OAuth2 en Open Banking, mécanisme par mécanisme

Le parcours de redirection du Jour 1, vu cette fois avec **les mécanismes de la S5** : on suit où interviennent

mTLS, **OAuth2**, **OpenID Connect** et **JWT** à chaque étape.



Qui émet le jeton, exactement ?

Le **jeton n'est jamais créé par le TPP**. Au terme de l'étape 5 (échange du code sur le canal mTLS), c'est la **banque (ASPSP)** qui le **forge et le signe**, puis le renvoie au TPP (étape 6). Le TPP ne fait que le *présenter* ensuite (Bearer) à chaque appel.

Deux jetons, deux rôles

L'**access token** est court (quelques minutes à quelques heures) et porte les scopes. Le **refresh token** sert à en obtenir un nouveau sans redemander la SCA — dans les limites du consentement et des règles d'authentification. Ne jamais confondre leur durée de vie. L'**ID token** (OIDC) est, lui, une preuve d'identité : il ne sert pas à appeler l'API, mais à savoir *qui* est connecté.

SÉQUENCE SUIVANTE · S6

Et les obligations réglementaires ?

Les mécanismes techniques (mTLS, OAuth2, OIDC, JWT) sont posés. Mais la **DSP2** ne se contente pas de « ça marche techniquement » : elle impose un cadre précis.

- **SCA renforcée** : authentification à plusieurs facteurs, appliquée aux moments clés (connexion, paiement, accès aux données).
- **Exemptions de SCA** : certains accès à faible risque en sont dispensés — sous conditions strictes.
- **Non-répudiation** : l'utilisateur ne peut pas nier une opération qu'il a validée (preuves conservées).
- **Horodatage** : chaque étape (consentement, autorisation, jeton) est datée et traçable.
- **Résilience & sécurité des API** : l'industrialisation de ces flux expose à l'usurpation, aux attaques d'API et aux fuites de données.

Autrement dit : la S5 explique *comment* prouver et autoriser ; la **S6 (Sécurité & conformité)** explique *quelles obligations* encadrent ces mécanismes et *quels risques* ils doivent couvrir.

06 · S6 · SÉCURITÉ & CONFORMITÉ

S6 - Sécurité & conformité

EXIGENCES DSP2 ET RISQUES API

Sécurité & conformité

La DSP2 ne dit pas seulement « protégez » : elle impose des mécanismes précis (SCA, exemptions, non-répudiation, horodatage) et laisse aux banques la responsabilité de tenir leurs APIs contre des menaces concrètes.

FIL ROUGE Lyam s'est authentifiée et l'application a son jeton. Mais que se passe-t-il si quelqu'un usurpe le TPP ? Si un appel est rejoué ? Si des données fuitent ? Cette séquence répond à ces menaces.

SCA

La Strong Customer Authentication

La **SCA** (authentification forte du client) exige au moins **deux facteurs indépendants** parmi trois catégories : **connaissance** (mot de passe, code), **possession** (téléphone, token), **inhérence** (empreinte, visage). Les deux facteurs doivent être de catégories différentes et leur compromission isolée ne doit pas suffire à contourner l'authentification.

Connaissance

Quelque chose que Lyam **sait** : mot de passe, code PIN.

Possession

Quelque chose que Lyam **possède** : son téléphone, une carte.

Inhérence

Quelque chose que Lyam **est** : empreinte digitale, reconnaissance faciale.

Quand la SCA s'applique

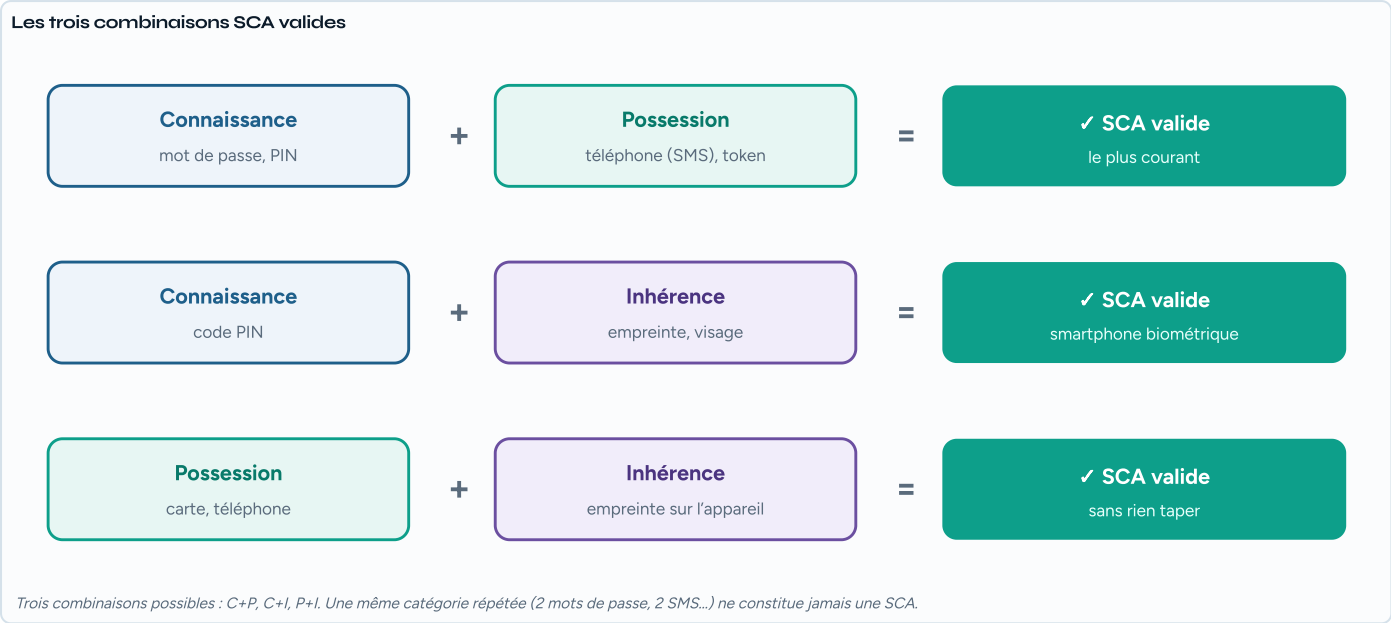
À l'accès initial aux comptes (AIS), puis lors des re-authentifications périodiques (au moins tous les 90 jours en AIS), et **à chaque paiement** initié par un PISP, sauf exemption applicable. Deux mots de passe ne font pas une SCA : il faut deux catégories différentes.

COMBINAISONS

Les combinaisons de facteurs possibles

La SCA exige **deux facteurs de catégories différentes**. Trois catégories existent : **connaissance** (ce que je sais), **possession** (ce que je possède) et **inhérence** (ce que je suis). On peut donc former **trois combinaisons**

valides — et toutes les combinaisons « deux fois la même catégorie » sont invalides.



Ce qui n'est PAS une SCA

- **Deux mots de passe** (2 × connaissance) → une seule catégorie, donc insuffisant.
- **Un mot de passe + une question secrète** (2 × connaissance) → insuffisant.
- **Deux codes SMS envoyés sur le même téléphone** (2 × possession) → un seul appareil compromis suffirait : insuffisant.
- Un facteur doit être **indépendant** : la compromission de l'un ne doit pas entraîner celle de l'autre.

MODES

Les différents modes de SCA en pratique

Une fois la combinaison choisie, plusieurs **modalités concrètes** existent, plus ou moins fluides. Voici les

schémas les plus répandus en Open Banking, classés par combinaison de facteurs.

MODE CONCRET	COMBINAISON	EXPÉRIENCE CLIENT	SCA ?
Mot de passe + code SMS	Connaissance + Possession	Saisie mot de passe puis code reçu par SMS	Oui
Mot de passe + application d'authentification (OTP)	Connaissance + Possession	Mot de passe puis code à usage unique généré par l'app	Oui
Push + empreinte (biométrie)	Possession + Inhérence	Notification poussée sur l'app, validée par empreinte	Oui
PIN + empreinte sur mobile	Connaissance + Inhérence	Code PIN puis reconnaissance du doigt	Oui
Carte + code confidentiel au terminal	**Possession + Connaissance**	**Carte insérée puis saisie du code au TPE**	Oui
Deux mots de passe	Connaissance + Connaissance	Deux secrets saisis	Non
Deux SMS sur le même téléphone	Possession + Possession	Deux codes reçus sur un seul appareil	Non

Le critère décisif : l'indépendance des facteurs

Ce qui compte n'est pas le *nombre* de manipulations, mais **l'indépendance des canaux** : si un même appareil ou un même secret peut compromettre les deux facteurs à la fois, la SCA n'est pas satisfaite. Un push + biométrie sur un mobile restent solides parce que le déverrouillage biométrique est local et lié à la personne.

EXEMPTIONS

Les exemptions à la SCA

La SCA systématique serait insupportable pour le client. Les RTS prévoient des exemptions, mais chacune est bornée et surveillée (fraude). La banque doit pouvoir justifier leur usage.

EXEMPTION	PRINCIPE	GARDE-FOU
Faible montant	Paiements de faible valeur sous un plafond	Compteur (nombre / cumul sur une période)
Bénéficiaire de confiance	Paiement vers un bénéficiaire déjà validé	SCA initiale à la création du bénéficiaire
Transaction récurrente	Paiements récurrents d'un même mandat	Premier paiement authentifié
Analyse du risque	Décision de l'ASPSP selon son propre scoring	Plafonds et taux de fraude surveillés
Accès aux soldes AIS	**Consultation après la SCA initiale**	**Jusqu'à 4 consultations/jour, SCA au moins tous les 90 jours**

Exemption ≠ absence de contrôle

Une exemption ne supprime pas la vérification : elle déplace la SCA à un autre moment (création du bénéficiaire, re-authentification périodique) et s'accompagne d'une surveillance de la fraude. Abuser d'une exemption entraîne son retrait.

PREUVE

Non-répudiation et horodatage

Non-répudiation

La banque doit pouvoir **prouver qu'un paiement ou un accès a bien eu lieu** et dans quelles conditions : qui, quel montant, quel bénéficiaire, à quel moment. Un client ne doit pas pouvoir nier un paiement qu'il a autorisé, et le TPP ne doit pas pouvoir contester un appel reçu.

Horodatage

Chaque événement porte une **date et heure de référence** fiable, conservée et cohérente entre les acteurs. C'est la base du contrôle : durée de validité d'un jeton, période de consentement, décompte des exemptions.

Ce que cela impose techniquement

Journalisation horodatée de chaque appel (requête/réponse), signature des messages pour garantir leur intégrité et leur origine (non-répudiation), conservation conforme aux délais légaux. Sans ces preuves, un litige ne peut être tranché.

RISQUES

Trois familles de risques API

Usurpation d'identité

Un attaquant se fait passer pour un TPP agréé ou pour le client.

Contre-mesures : mTLS strict, vérification du registre EBA, contrôle de cohérence certificat/agrément, SCA robuste.

Attaques API

Rejeu de requêtes, injection, déni de service, abus de quota.

Contre-mesures : anti-rejeu (nonce, horodatage), validation des entrées, throttling, quotas par TPP, surveillance d'anomalies.

Fuites de données

Exposition de soldes ou d'opérations au-delà du périmètre consenti, ou interception.

Contre-mesures : scopes minimaux, chiffrement en transit, contrôle de périmètre à chaque appel, pas de données sensibles dans les JWT.

RISQUE	POINT DE CONTRÔLE	MÉCANISME DU JOUR 2
Usurpation	Entrée de la chaîne	mTLS + registre EBA
Attaque API	Gateway & API Management	Throttling, anti-rejeu, validation
Fuite de données	**Services métier**	**Scopes + contrôle de consentement**

SÉQUENCE SUIVANTE

On met tout ensemble

Types d'API, composants, mécanismes d'authentification, exigences et risques : place au cas pratique. Ouvrir une API à un TPP, de bout en bout, avec le schéma, le flux OAuth et les points de contrôle.